FH-OÖ Hagenberg/HSD
SDP3, WS 2026
Übung 3

Name: Leander Kieweg, Julian Rieger — Aufwand in h: 14

Mat.Nr: S2510306014 S2510306023 — Punkte:

Übungsgruppe: 2 — korrigiert:

Beispiel 1 (24 Punkte) Player-Schnittstelle: Sie verwenden in Ihrer Firma HSDSoft einen MusicPlayer von der Firma MonkeySoft. Die öffentliche Schnittstelle des MusicPlayers sieht folgendermaßen aus und kann nicht verändert werden:

```
1     //starts playing with the actual song in list
2     void Start();
3     //stops playing
4     void Stop();
5     //switches to next song and starts at the end with first song
6     void SwitchNext();
7     //gets index of actual song
8     size_t GetActIndex() const;
9     //finds a song by name in playlist
10    bool Find(std::string const& name);
11    //gets count of songs in playlist
12    size_t GetCount() const;
13    //increases the volume relative to the actual volume
14    void IncreaseVol(size_t const vol);
15    //decreases the volume relative to the actual volume
16    void DecreaseVol(size_t const vol);
17    //adds a song to playlist
18    void Add(std::string const& name, size_t const dur);
```

Der MusicPlayer verwaltet Lieder und speichert den Namen und die Dauer jedes Liedes in Sekunden. Er kann gestartet und gestoppt werden und erlaubt das Verändern der Lautstärke. Die Lautstärke ist begrenzt mit 0 und maximal 100. Der Defaultwert für die Lautstärke liegt bei 15.

Zur Simulation liefert der Player je nach Aktion folgende Ausgaben auf der Konsole:

```
playing song number 1: Hells Bells (256 sec)
...
playing song number 4: Hawaguck (129 sec)
...
volume is now -> 70
song: Pulp Fiction not found!
stop song: Hells Bells (256 sec)
...
no song in playlist!
```

In weiterer Folge kaufen Sie einen VideoPlayer der Firma DonkeySoft mit folgender vorgegebenen Schnittstelle:

```
    //starts playing with the actual video in list
    void Play() const;
    //stops playing
    void Stop() const;
    //switches to first video in playlist and returns true, otherwise false if list is empty.
    bool First();
    //switches to next video in playlist and returns true, otherwise false if last song is reached.
    bool Next();
    //returns index of actual video
    size_t ActIndex() const;
    //returns name of actual video
    std::string const ActVideo() const;
    //sets volume (min volume=0 and max volume=50)
    void SetVolume(size_t const vol);
    //gets actual volume
    size_t GetVolume() const;
    //adds a video to playlist
    void Add(std::string const& name, size_t const dur, VideoFormat const& format);
}
```

Der VideoPlayer kann die Formate WMV, AVI und MKV abspielen. Er verwaltet Videos und speichert den Namen und die Dauer in Minuten. Er kann gestartet und gestoppt werden und erlaubt das Verändern der Lautstärke. Die Lautstärke ist begrenzt mit 0 und maximal 50. Der Defaultwert für die Lautstärke liegt bei 8.

Zur Simulation liefert der Player je nach Aktion folgende Ausgaben auf der Konsole:

```
playing video number 1: Die Sendung mit der Maus [duration -> 55 min], AVI-Format
...
playing video number 3: Freitag der 13te [duration -> 95 min], WMV-Format
...
volume is now -> 30
video: Hells Bells not found!
stop video: Pulp Fiction [duration -> 126 min], MKV-Format
...
no video in playlist!
```

Für einen Klienten soll nun nach außen folgende, unabhängige Schnittstelle zur Verfügung gestellt werden:

```
virtual void Play() = 0;
virtual void VolInc() = 0;
virtual void VolDec() = 0;
virtual void Stop() = 0;
virtual void Next() = 0;
virtual void Prev() = 0;
virtual void Select(std::string const& name) = 0;
```

Mit dieser Schnittstelle kann der Klient sowohl den MusicPlayer als auch den VideoPlayer verwenden. Die Methoden `VolInc()` und `VolDec()` erhöhen bzw. erniedrigen die Lautstärke um den Wert 1. `Next()` und `Prev()` schalten vor und zurück. `Select(...)` wählt ein Lied oder ein Video aus der Playliste aus und spielt es ab.

Implementieren Sie alle notwendigen Klassen (auch die Music/VideoPlayer-Klassen) und testen Sie diese entsprechend!

Implementieren Sie eine eigene `PlayerClient`-Klasse, die beide Player testet!

Achten Sie beim Design auf die Einhaltung der Design-Prinzipien und verwenden Sie ein entsprechendes Design-Pattern!

Allgemeine Hinweise: Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine **saubere Strukturierung** und auf eine **sorgfältige Ausarbeitung!** Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab!

HSD
OBERÖSTERREICH
FH-HAGENBERG

Systemdokumentation

Player-Schnittstelle

Version 1.0

J. Rieger, L. Kieweg

Hagenberg, 28. Oktober 2026

Dieses Dokument zeigt den prinzipiellen Aufbau einer Systemdokumentation für Software-Projekte. Die einzelnen Kapitel sind mit Kommentaren versehen, welche die Struktur und den Inhalt erläutern.

Inhaltsverzeichnis

1 Organisatorisches **2**
1.1 Team . . . 2
1.2 Aufteilung der Verantwortungsbereiche . . . 2
1.3 Aufwand . . . 2

2 Anforderungsdefinition (Systemspezifikation) **3**

3 Systementwurf **4**
3.1 Klassendiagramm . . . 4
3.2 Designentscheidungen . . . 5

4 Komponentenentwurf **5**

5 Testprotokollierung **6**
5.1 Ausgabe Konsole Testtreiber . . . 6

6 Quellcode **8**

1 Organisatorisches

1.1 Team

- Julian Rieger, Matr.-Nr: S2510306023, E-Mail: S2510306023@fhooe.at
- Leander Kieweg, Matr.-Nr: S2510306014, E-Mail: S2510306014@fhooe.at

1.2 Aufteilung der Verantwortungsbereiche

- Julian Rieger
  - Implementierung der Klassen: Video, PlayerClient, VideoPlayer, VideoAdapter
  - Implementierung des Testtreibers
  - Verfassen der Systemdokumentation
- Leander Kieweg
  - Implementierung der Klassen: Player, MusicAdapter, MusicPlayer, Media, Song
  - Design des Klassendiagrams
  - Implementierung des Testtreibers

1.3 Aufwand

- Julian Rieger: geschätzt: 7h, tatsächlich 7h
- Leander Kieweg: geschätzt 7h, tatsächlich 7h

2 Anforderungsdefinition (Systemspezifikation)

Es wird eine Software zur generischen Bedienung von zwei verschiedenen Schnittstellen, die eines Video Players und die eines Music Players, und stellt eine einzige, für beide verwendbare Schnittstelle, bereit. Die Schnittstelle soll sieben verschiedene Methoden zur Verfügung stellen, die die Geräte steuern können.

Funktionale Anforderungen

- **Play():** Startet die Wiedergabe des Titels.
- **Stop():** Beendet die Wiedergabe des Titels.
- **Next():** Springt in der Playlist auf den nächsten Titel.
- **Prev():** Springt in der Playlist auf den vorherige Titel.
- **VolInc():** Erhöht die Lautstärke um den Wert 1.
- **VolDec():** Verringert die Lautstärke um den Wert 1.
- **Select(const std::string name):** Wählt einen bestimmten, in der Playlist vorhandenen, Titel aus und startet die Wiedergabe.

Fehlerbehandlung

Fehler bei der Anwendung oder Bedienung des Programms werden abgefangen und es werden Rückmeldungen per std::cerr auf die Konsole ausgegeben und die Funktion wird mit return umgehend wieder verlassen. Zu diese Art von Fehlern zählen jegliche Fehlbedienungen der Schnittstelle. (z.B. Aufruf von Play bei leerer Playlist) Dabei kommt es zu keinen Einschränkungen der darauffolgenden Funktionsaufrufen, sofern ein korrekter Aufruf stattfindet.

3 Systementwurf

3.1 Klassendiagramm

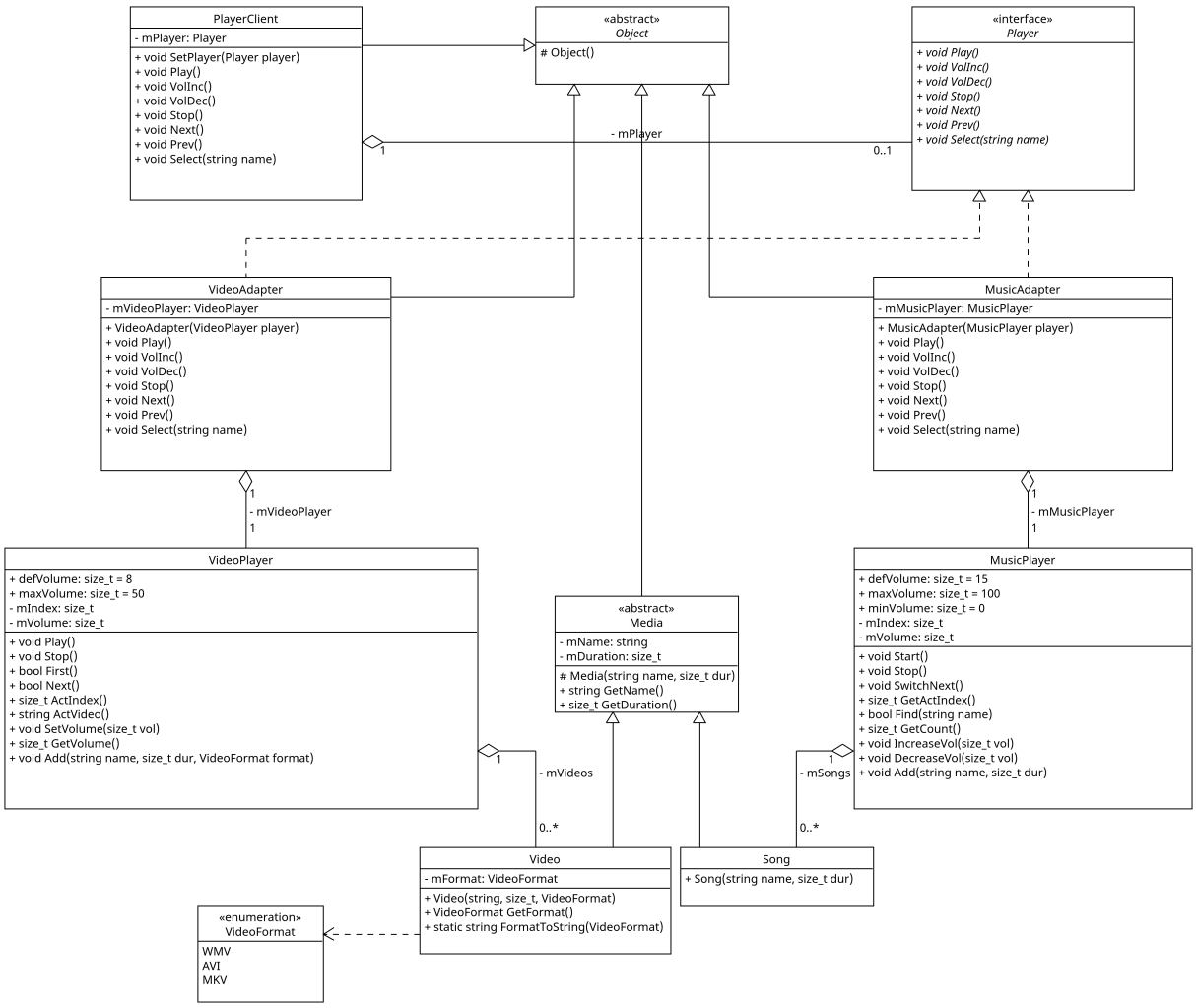


Abbildung 1: Klassendiagramm Player

Anmerkung: Das Klassendiagram steht für bessere Lesbarkeit ebenfalls im Sysdoku-Ordner zur Verfügung.

3.2 Designentscheidungen

Für die Konzeption und Implementierung der einheitlichen Player-Schnittstelle wurden gezielt objektorientierte Design-Prinzipien und Entwurfsmuster angewandt, um eine lose Kopplung und hohe Erweiterbarkeit zu garantieren.

Adapter-Pattern Da die beiden vorgegebenen Fremdklassen `VideoPlayer` (von DonkeySoft) und `MusicPlayer` (von MonkeySoft) unterschiedliche, unveränderliche Schnittstellen besitzen, wurde das **Adapter-Pattern** eingesetzt. Die Klassen `VideoAdapter` und `MusicAdapter` implementieren jeweils das einheitliche `Player`-Interface und übersetzen die Methodenaufrufe in die jeweilige proprietäre Schnittstelle. Dadurch kann der `PlayerClient` beide Medientypen über eine gemeinsame, generische Schnittstelle steuern.

Vererbungshierarchie und gemeinsame Basisklassen Zur Reduzierung von Code-Duplikaten und für ein sauberes Fundament erben alle zentralen Klassen von einer abstrakten Basisklasse `Object`. Zudem wurde für die Medienverwaltung die abstrakte Klasse `Media` eingeführt, von der die Klassen `Video` und `Song` abgeleitet sind. Gemeinsame Attribute wie Name und Dauer werden dadurch zentral in `Media` verwaltet.

Zustandsverwaltung und Kapselung Der `PlayerClient` hält eine optionale Referenz auf ein `Player`-Interface und delegiert Befehle direkt an diesen. Zur Vermeidung von Nullzeiger-Zugriffen wird im Client stets geprüft, ob ein gültiger Player gesetzt ist. Die spezifischen Restriktionen der Player (unterschiedliche Lautstärkebereiche wie 0–50 beim VideoPlayer bzw. 0–100 beim MusicPlayer) werden innerhalb der jeweiligen Adapter und Player sicher gekapselt und abgefangen.

Fehlerstrategie Fehlbedienungen (wie beispielsweise der Versuch, ein Medium bei leerer Playlist abzuspielen) werden kontrolliert abgefangen. Entsprechend den Vorgaben werden aussagekräftige Fehlermeldungen auf der Konsole ausgegeben und die betroffenen Methoden kontrolliert verlassen, ohne das Gesamtsystem zu destabilisieren.

4 Komponentenentwurf

Die HTML-Startdatei befindet sich im Verzeichnis ./../doxy/html/index.html

5 Testprotokollierung

5.1 Ausgabe Konsole Testtreiber

```
--- VideoPlayer ---
volume is now -> 5
Volume verified: 5
Index 0: Video1
playing video number 1: Video1 [duration -> 120 min], WMV-Format
Index 1: Video2
playing video number 2: Video2 [duration -> 90 min], AVI-Format
Index 2: Video3
playing video number 3: Video3 [duration -> 200 min], MKV-Format
stop video: Video3 [duration -> 200 min], MKV-Format
[SUCCESS] VideoPlayer passed

--- MusicPlayer ---
Count verified: 3
volume is now -> 18
volume is now -> 17
playing song number 1: Song1 (180 sec)
Index verified: 1
song: Song9 not found!
stop song: Song3 (150 sec)
[SUCCESS] MusicPlayer passed

--- VideoAdapter via Client ---
playing video number 1: Video1 [duration -> 120 min], WMV-Format
volume is now -> 9
playing video number 2: Video2 [duration -> 90 min], AVI-Format
playing video number 1: Video1 [duration -> 120 min], WMV-Format
playing video number 3: Video3 [duration -> 200 min], MKV-Format
video: NonExisting not found!
stop video: Video3 [duration -> 200 min], MKV-Format
--- MusicAdapter via Client ---
playing song number 1: Song1 (180 sec)
volume is now -> 14
playing song number 2: Song2 (210 sec)
playing song number 1: Song1 (180 sec)
playing song number 2: Song2 (210 sec)
song: NonExisting not found!
stop song: Song2 (210 sec)
[SUCCESS] Adapters passed

--- Edge Cases ---
no video in playlist!
no song in playlist!
[SUCCESS] EdgeCases passed
```

All tests successfully executed and verified!

6 Quellcode

Listing 1: ../Object.h

```
1  #ifndef OBJECT_H
2  #define OBJECT_H
3
4  /// @author Leander Kieweg
5  /// @date 27.10.2026
6  /// @brief Abstract base class of all classes.
7  class Object
8  {
9  public:
10     /// @brief Destroys the object.
11     virtual ~Object() = default;
12
13 protected:
14     /// @brief Creates an object. Only derived classes can call it.
15     Object() = default;
16 };
17
18 #endif //OBJECT_H
```

Listing 2: ../Media.h

```
1  #ifndef MEDIA_H
2  #define MEDIA_H
3  #include <cstddef>
4  #include <string>
5  #include "Object.h"
6
7  /// @author Leander Kieweg
8  /// @date 27.10.2026
9  /// @brief Abstract base class of all media with a name and a duration.
10 class Media : public Object
11 {
12 public:
13     /// @brief Name Getter
14     /// @returns the Name
15     const std::string& GetName() const;
16
17     /// @brief Duration Getter
18     /// @returns the Duration
19     size_t GetDuration() const;
20
21 protected:
22     /// @brief Creates a media.
23     /// @param[in] name is the name of the media.
24     /// @param[in] dur is the duration of the media.
25     Media(std::string const& name, size_t const dur);
26
27 private:
28     std::string mName;
29     size_t mDuration;
30 };
31
32 #endif //MEDIA_H
```

Listing 3: ../Media.cpp

```
1  #include "Media.h"
2
3  Media::Media(std::string const& name, size_t const dur) : mName(name), mDuration(
       dur) {
4
5  }
6
```

```
7  const std::string& Media::GetName() const {
8      return mName;
9  }
10
11 size_t Media::GetDuration() const {
12     return mDuration;
13 }
```

Listing 4: ../Video.h

```
1  #ifndef VIDEO_H
2  #define VIDEO_H
3  #include "Media.h"
4
5  /// @author Julian Rieger
6  /// @date 27.10.2026
7  /// @brief The formats a video can have.
8  enum class VideoFormat
9  {
10    WMV,
11    AVI,
12    MKV
13 };
14
15 /// @author Julian Rieger
16 /// @date 27.10.2026
17 /// @brief A video with a name, a duration and a format.
18 class Video : public Media
19 {
20 public:
21    /// @brief Creates a video.
22    /// @param[in] name is the name of the video.
23    /// @param[in] dur is the duration of the video.
24    /// @param[in] format is the format of the video.
25    Video(std::string const& name, size_t const dur, VideoFormat const& format);
26
27    /// @brief Returns mFormat
28    VideoFormat GetFormat() const;
29
30    static std::string FormatToString(VideoFormat format);
31
32 private:
33     VideoFormat mFormat;
34
35 };
36
37 #endif //VIDEO_H
```

Listing 5: ../Video.cpp

```
1  #include "Video.h"
2
3  Video::Video(std::string const& name, size_t const dur, VideoFormat const& format)
       :
4      Media(name, dur),
5      mFormat{ format }
6  {
7  }
8
9  VideoFormat Video::GetFormat() const
10 {
11     return mFormat;
12 }
13
14 std::string Video::FormatToString(VideoFormat format)
15 {
16     switch (format)
```

```
17      {
18          case VideoFormat::AVI: return "AVI";
19          case VideoFormat::MKV: return "MKV";
20          case VideoFormat::WMV: return "WMV";
21          default: return "NULL";
22      }
23  }
```

Listing 6: ../Song.h

```
1  #ifndef SONG_H
2  #define SONG_H
3  #include "Media.h"
4
5  /// @author Leander Kieweg
6  /// @date 27.10.2026
7  /// @brief A song with a name and a duration.
8  class Song : public Media
9  {
10 public:
11     /// @brief Creates a song.
12     /// @param[in] name is the name of the song.
13     /// @param[in] dur is the duration of the song.
14     Song(std::string const& name, size_t const dur);
15 };
16
17 #endif //SONG_H
```

Listing 7: ../Song.cpp

```
1  #include "Song.h"
2
3  Song::Song(std::string const& name, size_t const dur) : Media(name, dur) {
4
5  }
```

Listing 8: ../VideoPlayer.h

```
1  #ifndef VIDEOPLAYER_H
2  #define VIDEOPLAYER_H
3  #include <cstddef>
4  #include <string>
5  #include <vector>
6  #include "Video.h"
7
8  /// @author Julian Rieger
9  /// @date 27.10.2026
10 /// @brief Plays videos from a list.
11 class VideoPlayer
12 {
13 public:
14     /// @brief Starts playing the current video.
15     void Play() const;
16
17     /// @brief Stops playing.
18     void Stop() const;
19
20     /// @brief Selects the first video.
21     /// @return true if there is a first video.
22     bool First();
23
24     /// @brief Selects the next video.
25     /// @return true if there is a next video.
26     bool Next();
27
28     /// @brief Gets the index of the current video.
```

```
29      /// @return the index of the current video.
30      size_t ActIndex() const;
31
32      /// @brief Gets the name of the current video.
33      /// @return the name of the current video.
34      std::string const ActVideo() const;
35
36      /// @brief Sets the volume.
37      /// @param[in] vol is the new volume.
38      void SetVolume(size_t const vol);
39
40      /// @brief Gets the volume.
41      /// @return the current volume.
42      size_t GetVolume() const;
43
44      /// @brief Adds a video to the list.
45      /// @param[in] name is the name of the video.
46      /// @param[in] dur is the duration of the video.
47      /// @param[in] format is the format of the video.
48      void Add(std::string const& name, size_t const dur, VideoFormat const& format);
49
50      static constexpr size_t defVolume = 8;
51      static constexpr size_t maxVolume = 50;
52
53  private:
54       std::vector<Video> mVideos;
55      size_t mIndex{ 0 };
56      size_t mVolume{ defVolume };
57  };
58
59  #endif //VIDEOPLAYER_H
```

Listing 9: ../VideoPlayer.cpp

```
1   #include "VideoPlayer.h"
2   #include <iostream>
3   #include <stdexcept>
4
5
6   void VideoPlayer::Play() const {
7       if (mVideos.empty()) {
8           std::cerr << "no video in playlist!" << std::endl;
9           return;
10      }
11      const auto& vid = mVideos.at(mIndex);
12      std::cout << "playing video number " << (mIndex + 1) << ": " << vid.GetName()
13                << " [duration -> " << vid.GetDuration() << " min], "
14                << Video::FormatToString(vid.GetFormat()) << "-Format" << std::endl;
15  }
16
17  void VideoPlayer::Stop() const {
18      if (mVideos.empty()) {
19          std::cerr << "no video in playlist!" << std::endl;
20          return;
21      }
22      const auto& vid = mVideos.at(mIndex);
23      std::cout << "stop video: " << vid.GetName()
24                << " [duration -> " << vid.GetDuration() << " min], "
25                << Video::FormatToString(vid.GetFormat()) << "-Format" << std::endl;
26  }
27
28  bool VideoPlayer::First() {
29      if (mVideos.empty()) {
30          return false;
31      }
32      mIndex = 0;
33      return true;
34  }
35
```

```
36 bool VideoPlayer::Next() {
37     if (mVideos.empty() || mIndex + 1 >= mVideos.size()) {
38         return false;
39     }
40     mIndex++;
41     return true;
42 }
43
44 size_t VideoPlayer::ActIndex() const
45 {
46     return mIndex;
47 }
48
49 std::string const VideoPlayer::ActVideo() const
50 {
51     return mVideos.at(mIndex).GetName();
52 }
53
54 void VideoPlayer::SetVolume(size_t const vol) {
55     if (vol > maxVolume) {
56         mVolume = maxVolume;
57     } else {
58         mVolume = vol;
59     }
60     std::cout << "volume is now -> " << mVolume << std::endl;
61 }
62
63 size_t VideoPlayer::GetVolume() const
64 {
65     return mVolume;
66 }
67
68 void VideoPlayer::Add(std::string const& name, size_t const dur, VideoFormat const
      & format)
69 {
70     mVideos.push_back({ name, dur, format });
71 }
```

Listing 10: ../VideoAdapter.h

```
1  #ifndef VIDEOADAPTER_H
2  #define VIDEOADAPTER_H
3  #include "Object.h"
4  #include "Player.h"
5  #include "VideoPlayer.h"
6
7  /// @author Julian Rieger
8  /// @date 27.10.2026
9  /// @brief Makes the VideoPlayer usable as a Player.
10 class VideoAdapter : public Object, public Player
11 {
12 public:
13    /// @brief Constructs a new VideoAdapter.
14    /// @param[in] player is the VideoPlayer to adapt.
15    explicit VideoAdapter(VideoPlayer& player);
16
17    /// @brief Starts playing.
18    void Play() override;
19
20    /// @brief Increases the volume.
21    void VolInc() override;
22
23    /// @brief Decreases the volume.
24    void VolDec() override;
25
26    /// @brief Stops playing.
27    void Stop() override;
28
29    /// @brief Switches to the next video.
```

```
30      void Next() override;
31
32      /// @brief Switches to the previous video.
33      void Prev() override;
34
35      /// @brief Selects a video by its name.
36      /// @param[in] name is the name of the video.
37      void Select(std::string const& name) override;
38
39  private:
40      VideoPlayer& mVideoPlayer;
41  };
42
43  #endif //VIDEOADAPTER_H
```

Listing 11: ../VideoAdapter.cpp

```
1   #include "VideoAdapter.h"
2   #include <iostream>
3
4   VideoAdapter::VideoAdapter(VideoPlayer& player) : mVideoPlayer{ player } {
5
6   }
7
8   void VideoAdapter::Play()
9   {
10      mVideoPlayer.Play();
11  }
12
13  void VideoAdapter::VolDec() {
14      size_t vol = mVideoPlayer.GetVolume();
15      if (vol > 0) {
16          mVideoPlayer.SetVolume(vol - 1);
17      }
18  }
19
20  void VideoAdapter::VolInc() {
21      size_t vol = mVideoPlayer.GetVolume();
22      if (vol < VideoPlayer::maxVolume) {
23          mVideoPlayer.SetVolume(vol + 1);
24      }
25  }
26
27  void VideoAdapter::Stop()
28  {
29      mVideoPlayer.Stop();
30  }
31
32  void VideoAdapter::Next()
33  {
34      mVideoPlayer.Next();
35  }
36
37  void VideoAdapter::Prev() {
38      if (mVideoPlayer.ActIndex() > 0) {
39          size_t targetIndex = mVideoPlayer.ActIndex() - 1;
40          mVideoPlayer.First();
41          for (size_t i = 0; i < targetIndex; ++i) {
42              mVideoPlayer.Next();
43          }
44      }
45  }
46
47  void VideoAdapter::Select(std::string const& name) {
48      if (!mVideoPlayer.First()) {
49          std::cerr << "video: " << name << " not found!" << std::endl;
50          return;
51      }
52
```

```
53      do {
54          if (mVideoPlayer.ActVideo() == name) {
55              Play();
56              return;
57          }
58      } while (mVideoPlayer.Next());
59
60      std::cerr << "video: " << name << " not found!" << std::endl;
61  }
```

Listing 12: ../MusicPlayer.h

```
1   #ifndef MUSICPLAYER_H
2   #define MUSICPLAYER_H
3   #include <cstddef>
4   #include <string>
5   #include <vector>
6   #include "Song.h"
7
8   /// @author Leander Kieweg
9   /// @date 27.10.2026
10  /// @brief Plays songs from a list.
11  class MusicPlayer
12  {
13  public:
14      /// @brief Starts playing the current song.
15      void Start();
16
17      /// @brief Stops playing.
18      void Stop();
19
20      /// @brief Switches to the next song.
21      void SwitchNext();
22
23      /// @brief Gets the index of the current song.
24      /// @return the index of the current song.
25      size_t GetActIndex() const;
26
27      /// @brief Searches a song by its name and selects it.
28      /// @param[in] name is the name of the song.
29      /// @return true if the song was found.
30      bool Find(std::string const& name);
31
32      /// @brief Gets the number of songs.
33      /// @return the number of songs.
34      size_t GetCount() const;
35
36      /// @brief Increases the volume.
37      /// @param[in] vol is the value to increase by.
38      void IncreaseVol(size_t const vol);
39
40      /// @brief Decreases the volume.
41      /// @param[in] vol is the value to decrease by.
42      void DecreaseVol(size_t const vol);
43
44      /// @brief Adds a song to the list.
45      /// @param[in] name is the name of the song.
46      /// @param[in] dur is the duration of the song.
47      void Add(std::string const& name, size_t const dur);
48
49  private:
50      static constexpr size_t defVolume = 15;
51      static constexpr size_t maxVolume = 100;
52      static constexpr size_t minVolume = 0;
53
54      size_t mVolume{ defVolume };
55      size_t mIndex{ 0 };
56      std::vector<Song> mSongs;
57  };
```

```
58
59 #endif //MUSICPLAYER_H
```

Listing 13: ../MusicPlayer.cpp

```
1  #include "MusicPlayer.h"
2  #include <algorithm>
3  #include <iostream>
4
5  void MusicPlayer::Start() {
6      if (mSongs.empty()) {
7          std::cerr << "no song in playlist!" << std::endl;
8          return;
9      }
10
11     if (mIndex >= mSongs.size()) {
12         mIndex = 0;
13     }
14
15     const Song& song = mSongs[mIndex];
16     std::cout << "playing song number " << mIndex + 1 << ": "
17               << song.GetName() << " (" << song.GetDuration() << " sec)"
18               << std::endl;
19 }
20
21 void MusicPlayer::Stop() {
22     if (mSongs.empty()) {
23         std::cerr << "no song in playlist!" << std::endl;
24         return;
25     }
26
27     if (mIndex >= mSongs.size()) {
28         mIndex = 0;
29     }
30
31     const Song& song = mSongs[mIndex];
32     std::cout << "stop song: " << song.GetName() << " ("
33               << song.GetDuration() << " sec)" << std::endl;
34 }
35
36 void MusicPlayer::SwitchNext() {
37     if (mSongs.empty()) {
38         return;
39     }
40
41     mIndex = (mIndex + 1) % mSongs.size();
42 }
43
44 size_t MusicPlayer::GetActIndex() const {
45     return mIndex;
46 }
47
48 bool MusicPlayer::Find(std::string const& name) {
49     auto it = std::ranges::find_if(mSongs,
50         [&name](Song const& song) {
51             return song.GetName() == name;
52         });
53
54     if (it == mSongs.end()) {
55         std::cerr << "song: " << name << " not found!" << std::endl;
56         return false;
57     }
58
59     mIndex = static_cast<size_t>(std::distance(mSongs.begin(), it));
60     return true;
61 }
62
63 size_t MusicPlayer::GetCount() const {
64     return mSongs.size();
```

```
65 }
66
67 void MusicPlayer::IncreaseVol(size_t const vol) {
68     if (vol >= maxVolume - mVolume) {
69         mVolume = maxVolume;
70     }
71     else {
72         mVolume += vol;
73     }
74     std::cout << "volume is now -> " << mVolume << std::endl;
75 }
76
77 void MusicPlayer::DecreaseVol(size_t const vol) {
78     if (vol >= mVolume) {
79         mVolume = minVolume;
80     }
81     else {
82         mVolume -= vol;
83     }
84     std::cout << "volume is now -> " << mVolume << std::endl;
85 }
86
87 void MusicPlayer::Add(std::string const& name, size_t const dur) {
88     mSongs.push_back({name, dur});
89 }
```

Listing 14: ../MusicAdapter.h

```
1  #ifndef MUSICADAPTER_H
2  #define MUSICADAPTER_H
3  #include "Object.h"
4  #include "Player.h"
5  #include "MusicPlayer.h"
6
7  /// @author Leander Kieweg
8  /// @date 27.10.2026
9  /// @brief Makes the MusicPlayer usable as a Player.
10 class MusicAdapter : public Object, public Player
11 {
12 public:
13     /// @brief Constructs a new MusicAdapter.
14     /// @param[in] player is the MusicPlayer to adapt.
15     explicit MusicAdapter(MusicPlayer& player);
16
17     /// @brief Starts playing.
18     void Play() override;
19
20     /// @brief Increases the volume.
21     void VolInc() override;
22
23     /// @brief Decreases the volume.
24     void VolDec() override;
25
26     /// @brief Stops playing.
27     void Stop() override;
28
29     /// @brief Switches to the next song.
30     void Next() override;
31
32     /// @brief Switches to the previous song.
33     void Prev() override;
34
35     /// @brief Selects a song by its name.
36     /// @param[in] name is the name of the song.
37     void Select(std::string const& name) override;
38
39 private:
40     MusicPlayer& mMusicPlayer;
41 };
```

```
42
43 #endif //MUSICADAPTER_H
```

Listing 15: ../MusicAdapter.cpp

```
1  #include "MusicAdapter.h"
2  #include <iostream>
3
4  MusicAdapter::MusicAdapter(MusicPlayer& player) : mMusicPlayer{ player } {
5
6  }
7
8  void MusicAdapter::Play()
9  {
10     mMusicPlayer.Start();
11 }
12
13 void MusicAdapter::VolInc()
14 {
15     mMusicPlayer.IncreaseVol(1);
16 }
17
18 void MusicAdapter::VolDec()
19 {
20     mMusicPlayer.DecreaseVol(1);
21 }
22
23 void MusicAdapter::Stop()
24 {
25     mMusicPlayer.Stop();
26 }
27
28 void MusicAdapter::Next()
29 {
30     mMusicPlayer.SwitchNext();
31 }
32
33 void MusicAdapter::Prev()
34 {
35     const size_t count = mMusicPlayer.GetCount();
36     if (count == 0) {
37         return;
38     }
39
40     const size_t current = mMusicPlayer.GetActIndex();
41     size_t target = count - 1;
42     if (current > 0 && current < count) {
43         target = current - 1;
44     }
45
46     while (mMusicPlayer.GetActIndex() != target) {
47         mMusicPlayer.SwitchNext();
48     }
49 }
50
51 void MusicAdapter::Select(std::string const& name) {
52     if (mMusicPlayer.Find(name)) {
53         mMusicPlayer.Start();
54     }
55 }
```

Listing 16: ../Player.h

```
1  #ifndef PLAYER_H
2  #define PLAYER_H
3  #include <string>
4
```

```
5  /// @author Leander Kieweg
6  /// @date 27.10.2026
7  /// @brief Interface for all players.
8  class Player
9  {
10 public:
11    /// @brief Destroys the player.
12    virtual ~Player() = default;
13
14    /// @brief Starts playing.
15    virtual void Play() = 0;
16
17    /// @brief Increases the volume.
18    virtual void VolInc() = 0;
19
20    /// @brief Decreases the volume.
21    virtual void VolDec() = 0;
22
23    /// @brief Stops playing.
24    virtual void Stop() = 0;
25
26    /// @brief Switches to the next entry.
27    virtual void Next() = 0;
28
29    /// @brief Switches to the previous entry.
30    virtual void Prev() = 0;
31
32    /// @brief Selects an entry by its name.
33    /// @param[in] name is the name of the entry.
34    virtual void Select(std::string const& name) = 0;
35 };
36
37 #endif //PLAYER_H
```

Listing 17: ../PlayerClient.h

```
1  #ifndef PLAYERCLIENT_H
2  #define PLAYERCLIENT_H
3  #include "Object.h"
4  #include "Player.h"
5
6  /// @author Julian Rieger
7  /// @date 27.10.2026
8  /// @brief Uses a player through the Player interface.
9  class PlayerClient : public Object
10 {
11 public:
12
13    /// @brief Sets the player to use.
14    /// @param[in] player is the player to use.
15    void SetPlayer(Player& player);
16
17    /// @brief Starts playing.
18    void Play();
19
20    /// @brief Increases the volume.
21    void VolInc();
22
23    /// @brief Decreases the volume.
24    void VolDec();
25
26    /// @brief Stops playing.
27    void Stop();
28
29    /// @brief Switches to the next entry.
30    void Next();
31
32    /// @brief Switches to the previous entry.
33    void Prev();
```

```
34
35      /// @brief Selects an entry by its name.
36      /// @param[in] name is the name of the entry.
37      void Select(std::string const& name);
38
39  private:
40      Player* mPlayer{ nullptr };
41
42  };
43
44  #endif //PLAYERCLIENT_H
```

Listing 18: ../PlayerClient.cpp

```
1   #include "PlayerClient.h"
2   #include <iostream>
3
4   void PlayerClient::SetPlayer(Player& player)
5   {
6       mPlayer = &player;
7   }
8
9   void PlayerClient::Play()
10  {
11      if (mPlayer != nullptr)
12      {
13          mPlayer->Play();
14      }
15  }
16
17  void PlayerClient::VolInc()
18  {
19      if (mPlayer != nullptr)
20      {
21          mPlayer->VolInc();
22      }
23  }
24
25  void PlayerClient::VolDec()
26  {
27      if (mPlayer != nullptr)
28      {
29          mPlayer->VolDec();
30      }
31  }
32
33  void PlayerClient::Stop()
34  {
35      if (mPlayer != nullptr)
36      {
37          mPlayer->Stop();
38      }
39  }
40
41  void PlayerClient::Next()
42  {
43      if (mPlayer != nullptr)
44      {
45          mPlayer->Next();
46      }
47  }
48
49  void PlayerClient::Prev()
50  {
51      if (mPlayer != nullptr)
52      {
53          mPlayer->Prev();
54      }
55  }
```

```
56
57 void PlayerClient::Select(std::string const& name)
58 {
59     if (mPlayer != nullptr)
60     {
61         mPlayer->Select(name);
62     }
63 }
```

Listing 19: ../main.cpp

```
1  /// @mainpage Player Adapter Test Driver
2  /// @brief Test driver for verifying the Player Adapter Pattern implementation
       with assertions.
3  ///
4  /// Tests the direct usage of VideoPlayer and MusicPlayer, as well as their
5  /// unified access through the PlayerClient using the Adapter Pattern (
       VideoAdapter
6  /// and MusicAdapter). Additionally, edge cases such as empty playlists and
7  /// uninitialized client pointers are verified via assertions.
8  ///
9  /// @author Leander Kieweg, Julian Rieger
10 /// @date 28.10.2026
11 /// @version 1.1
12
13 #include <iostream>
14 #include <string>
15 #include <cassert>
16 #include "PlayerClient.h"
17 #include "VideoAdapter.h"
18 #include "MusicAdapter.h"
19 #include "VideoPlayer.h"
20 #include "MusicPlayer.h"
21
22 /// @brief Tests the VideoPlayer directly and asserts expected states.
23 static void TestVideoPlayer()
24 {
25     std::cout << "--- VideoPlayer ---" << std::endl;
26     VideoPlayer player;
27
28     player.Add("Video1", 120, VideoFormat::WMV);
29     player.Add("Video2", 90, VideoFormat::AVI);
30     player.Add("Video3", 200, VideoFormat::MKV);
31
32     player.SetVolume(5);
33     assert(player.GetVolume() == 5); // Prüfung: Lautstärke korrekt gesetzt?
34     std::cout << "Volume verified: " << player.GetVolume() << std::endl;
35
36     bool hasFirst = player.First();
37     assert(hasFirst == true); // Prüfung: Playlist ist nicht leer
38
39     size_t expectedIndex = 0;
40     do
41     {
42         assert(player.ActIndex() == expectedIndex); // Prüfung: Index-Ablauf
               stimmt
43         std::cout << "Index " << player.ActIndex() << ": " << player.ActVideo() <<
                std::endl;
44         player.Play();
45         expectedIndex++;
46     } while (player.Next());
47
48     assert(player.ActIndex() == 2); // Prüfung: Am Ende des Vectors angelangt
49     player.Stop();
50
51     std::cout << "[SUCCESS] VideoPlayer passed" << std::endl << std::endl;
52 }
53
54 /// @brief Tests the MusicPlayer directly and asserts expected states.
```

```
55 static void TestMusicPlayer()
56 {
57     std::cout << "--- MusicPlayer ---" << std::endl;
58     MusicPlayer player;
59 
60     player.Add("Song1", 180);
61     player.Add("Song2", 210);
62     player.Add("Song3", 150);
63     assert(player.GetCount() == 3); // Prüfung: Anzahl der Songs korrekt
64     std::cout << "Count verified: " << player.GetCount() << std::endl;
65 
66     player.IncreaseVol(3);
67     player.DecreaseVol(1);
68 
69     player.Start();
70     assert(player.GetActIndex() == 0); // Prüfung: Startindex ist 0
71 
72     player.SwitchNext();
73     assert(player.GetActIndex() == 1); // Prüfung: Wechsel auf Index 1 erfolgreich
74     std::cout << "Index verified: " << player.GetActIndex() << std::endl;
75 
76     assert(player.Find("Song3") == true);  // Prüfung: Vorhandener Song gefunden
77     assert(player.Find("Song9") == false); // Prüfung: Nicht vorhandener Song
           liefert false
78 
79     player.Stop();
80     std::cout << "[SUCCESS] MusicPlayer passed" << std::endl << std::endl;
81 }
82 
83 /// @brief Tests both adapters through the PlayerClient.
84 static void TestAdapters()
85 {
86     PlayerClient client;
87 
88     std::cout << "--- VideoAdapter via Client ---" << std::endl;
89     VideoPlayer videoPlayer;
90     videoPlayer.Add("Video1", 120, VideoFormat::WMV);
91     videoPlayer.Add("Video2", 90, VideoFormat::AVI);
92     videoPlayer.Add("Video3", 200, VideoFormat::MKV);
93 
94     VideoAdapter videoAdapter(videoPlayer);
95     client.SetPlayer(videoAdapter);
96 
97     client.Play();
98     client.VolInc();
99     client.Next();
100    client.Play();
101    client.Prev();
102    client.Play();
103    client.Select("Video3");
104    client.Select("NonExisting");
105    client.Stop();
106 
107    std::cout << "--- MusicAdapter via Client ---" << std::endl;
108    MusicPlayer musicPlayer;
109    musicPlayer.Add("Song1", 180);
110    musicPlayer.Add("Song2", 210);
111    musicPlayer.Add("Song3", 150);
112 
113    MusicAdapter musicAdapter(musicPlayer);
114    client.SetPlayer(musicAdapter);
115 
116    client.Play();
117    client.VolDec();
118    client.Next();
119    client.Play();
120    client.Prev();
121    client.Play();
122    client.Select("Song2");
123    client.Select("NonExisting");
```

```
124     client.Stop();
125
126     std::cout << "[SUCCESS] Adapters passed" << std::endl << std::endl;
127 }
128
129 /// @brief Tests edge cases like nullptr handling and empty playlists with
        assertions.
130 static void TestEdgeCases()
131 {
132     std::cout << "--- Edge Cases ---" << std::endl;
133
134     // Test uninitialized client pointer (should not crash)
135     PlayerClient emptyClient;
136     emptyClient.Play();
137
138     // Test empty VideoPlayer behavior
139     VideoPlayer emptyVideoPlayer;
140     emptyVideoPlayer.Play();
141     assert(emptyVideoPlayer.First() == false); // Prüfung: Leere Playlist gibt
            false zurück
142
143     // Test empty MusicPlayer behavior
144     MusicPlayer emptyMusicPlayer;
145     emptyMusicPlayer.Start();
146     assert(emptyMusicPlayer.GetCount() == 0); // Prüfung: Zähler ist 0
147
148     std::cout << "[SUCCESS] EdgeCases passed" << std::endl << std::endl;
149 }
150
151 int main()
152 {
153     TestVideoPlayer();
154     TestMusicPlayer();
155     TestAdapters();
156     TestEdgeCases();
157
158     std::cout << "All tests successfully executed and verified!" << std::endl;
159     return 0;
160 }
```